TECHNICAL UNIVERSITY OF MOLDOVA
Faculty of Computers, Informatics, and Microelectronics
Department of Software Engineering
Software Engineering Study Programme

LABORATORY REPORT

Lab 7: Three Small Programs: Processes, Redirection, a Pipe

Operating Systems

Student: Gafenco Victor

Group: FAF-241

Instructor: Assistant lecturer, Patricia Reițman

Chișinău - 2026

CONTENTS

Abstract **3**

Introduction **4**

1 Process creation and program replacement **5**

1.1 The three steps 5

1.2 Observed successful and failed execution 5

2 A zombie process **6**

3 Output redirection **7**

4 A two-process pipeline **8**

4.1 Descriptor ownership and result 8

4.2 Keeping a writer open 8

Conclusions **9**

ABSTRACT

Four Linux programs demonstrate process creation, program replacement, delayed child reaping, standard-output redirection and a two-stage pipeline. Successful and failed execution, a visible zombie and a deliberately retained pipe writer connect the observed behaviour to the underlying process and descriptor operations.

Keywords: Linux, fork, exec, waitpid, zombie, redirection, pipe

INTRODUCTION

Four C programs examine process creation, program replacement, child reaping, output redirection and communication through a pipe on Linux. The submitted `run.c` runs `ls -l`; the `date` and nonexistent-program versions were also compiled and executed. `zombie.c` delays reaping by 20 seconds, `redirect.c` sends a listing to a file, and `pipeline.c` connects `ls /etc` to `wc -l`.

All four programs compiled without diagnostics using GCC (SUSE Linux) 16.2.1 with `-Wall -Wextra -Werror`. Each can be built independently, for example:

```
gcc -Wall -Wextra -o run run.c
./run
```

The complete command outputs are in `evidence/`. Process identifiers, directory contents and timestamps describe these runs and can differ on another machine. The experiments assume standard descriptors 0, 1 and 2 are initially open.

1 PROCESS CREATION AND PROGRAM REPLACEMENT

1.1 The three steps

`fork()` creates a child and returns its PID to the parent, zero to the child, or a negative value on failure. Both processes continue after the call. In the child, `execlp()` searches for an executable using `PATH` and replaces the current program image. The process retains its PID. The parent calls `waitpid(pid, &status, 0)` to collect this particular child's termination status.

The initial `fflush(stdout)` prevents buffered parent output from being copied and printed twice after `fork`. The child's flush preserves its diagnostic before `exec` replaces its address space. The final parent message follows the child's termination because of `waitpid`.

1.2 Observed successful and failed execution

Selected output from `evidence/run.txt`:

```
$ ./run
parent: my PID is 27
child:  my PID is 28, my parent is 27
total 108
[listing omitted]
parent: child 28 ended with status 0
```

Replacing the execution line with `execlp("date", "date", (char *)NULL);` and recompiling produced:

```
parent: my PID is 29
child:  my PID is 35, my parent is 29
Mon Oct 26 12:01:04 EET 2026
parent: child 35 ended with status 0
```

Replacing it with `execlp("nosuchprog", "nosuchprog", (char *)NULL);` produced:

```
parent: my PID is 36
child:  my PID is 42, my parent is 36
exec: No such file or directory
parent: child 42 ended with status 127
```

1. `perror("exec")` runs only after a failed `execlp`: successful execution never returns to the old program. The nonexistent executable causes the call to return with an error.
2. The child printed 27 as "my parent" in the first run. This is the original process's PID, returned in the child by `getppid()`, and it matches the parent's first message.
3. The parent observed status 0 for `ls` and 127 for the failed execution. The latter comes from the explicit `exit(127)` after `perror`, not directly from `execlp`. The `date` case also returned 0.

Linux encodes the wait status. These children exited normally, so `WEXITSTATUS(status)` extracts their exit codes; signal termination would require separate handling.

2 A ZOMBIE PROCESS

`zombie.c` is the original `ls` version with `sleep(20);` immediately before `waitpid`. During this interval, `ls` finishes but the parent has not yet collected its status. The process table was filtered to the two relevant PIDs:

```
$ gcc -Wall -Wextra -o zombie zombie.c
$ ./zombie > /dev/null &
$ sleep 2
$ ps -o pid,ppid,stat,cmd -p 43,44
    PID    PPID STAT CMD
     43       2 S    ./zombie
     44      43 Z    [ls] <defunct>
```

After the parent's 20-second sleep and subsequent wait:

```
$ ps -o pid,ppid,stat,cmd -p 43,44
    PID    PPID STAT CMD
```

1. `ls` has state `Z`, meaning zombie: it has terminated, but its exit information remains available for the parent. It is no longer executing instructions. The parent is sleeping (`S`).
2. The `[ls] <defunct>` entry is absent after the wait. The line `waitpid(pid, &status, 0);` reaps the child. `sleep(20)` only delays that operation; it does not reap anything. The parent also exits after printing its final message.

3 OUTPUT REDIRECTION

In the child, `close(1)` releases standard output. `open("out.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644)` opens or creates the destination and truncates any previous contents. Its requested creation permissions are filtered by the process's umask. The diagnostic uses standard error, so it remains visible on the terminal.

```
$ gcc -Wall -Wextra -o redirect redirect.c
$ ./redirect
open gave descriptor 1
done: the listing is in out.txt
$ head -3 out.txt
total 108
-rw-r--r--. 1 victor victor     0 Oct 26 12:01 out.txt
-rwxr-xr-x. 1 victor victor 20456 Oct 26 12:01 pipeline
```

1. `open` returned 1 because it allocates the lowest unused descriptor, and the child had just closed 1 while 0 and 2 remained open.
2. `ls` still writes to descriptor 1. Open descriptors survive `exec` unless marked close-on-exec, so descriptor 1 now identifies `out.txt`. The parent's separate descriptor table still directs its "done" message to the terminal.
3. To redirect errors with the same technique, close descriptor 2 before opening `err.txt`; the newly opened file then becomes standard error.

The zero size shown for `out.txt` within its own listing reflects the metadata read by `ls` before its buffered listing was written. Reading the file afterward confirms that it contains output.

4 A TWO-PROCESS PIPELINE

4.1 Descriptor ownership and result

`pipe(p)` supplies a read end `p[0]` and a write end `p[1]`. Both children inherit them. The `ls` child closes descriptor 1 and calls `dup(p[1])`, making standard output refer to the pipe. The `wc` child closes descriptor 0 and calls `dup(p[0])`, making standard input refer to the pipe. Both children close the original pipe descriptors before `exec`. The parent closes both ends and waits twice.

```
gcc -Wall -Wextra -o pipeline pipeline.c
$ ./pipeline
213
$ ls /etc | wc -l
213
```

The two numbers were equal: both commands counted 213 output lines from `ls /etc` in the test environment.

4.2 Keeping a writer open

The parent's `close(p[1]);` was commented out in a separate experimental version and the program was recompiled. It produced no output and remained running. The process table after two seconds showed:

```
$ ./pipeline-broken
(no output; still running after 2 seconds)
$ ps -o pid,ppid,stat,cmd -p 61,63
    PID    PPID STAT CMD
     61       2 Ss   ./pipeline-broken
     63      61 S    wc -l
```

The experiment was stopped with SIGINT, the signal produced by Ctrl-C. The submitted `pipeline.c` retains the correct close.

1. The working program and the shell both printed 213.
2. `wc` is the child that never finishes. `ls` has already exited, while the parent is waiting for `wc`.
3. `wc` is waiting for end-of-file, which cannot arrive while the parent still holds an open reference to the pipe's write end.

An empty pipe is not necessarily at EOF: a blocking read waits if any writer remains. Closing unused descriptors is therefore part of the communication protocol, not merely housekeeping.

CONCLUSIONS

The experiments demonstrate how a shell combines process and descriptor operations. `fork` creates a child, `exec` replaces its program, and `waitpid` collects its result. Delaying the wait exposes a zombie. Redirection works by changing the child's descriptor table before execution, and a pipeline connects two otherwise unchanged programs. The stalled-pipe experiment confirms that EOF depends on closing every reference to the writing end.